# Cloud  ASSIGNMENT - 3

## Q1. Docker Volume Persistence

Create a Docker volume named
data_vol.

```
mojorojo@mojorojo-vmware:~/dock-her$ docker volume create data_vol
data_vol
```

Run a container using this volume mounted at
Create a file inside /data.

```
mojorojo@mojorojo-vmware:~/dock-her$ docker volume ls
DRIVER    VOLUME NAME
local     2e66e3ac38a957ded238754f1dab9e687ef0a38391ec0fd0e892d82b1bb86fa4
local     8f27e14771fe7dc6a1596df8f1564bdc786a97db0b702c4983c58fb6e7215fa6
local     53090204c0a630421606b35c179648baba408fce0ecf6ad7a97ea1a4f4737477
local     data_vol
mojorojo@mojorojo-vmware:~/dock-her$ docker run -it --name container1 -v data_vol:/data ubuntu /bin/bash
root@f4c1647c9622:/# cd /d
data/ dev/
root@f4c1647c9622:/# cd /d
data/ dev/
root@f4c1647c9622:/# cd /data/
root@f4c1647c9622:/data# touch hello.txt
root@f4c1647c9622:/data# echo "This is from Container 1" >> hello.txt
root@f4c1647c9622:/data# exit
exit
mojorojo@mojorojo-vmware:~/dock-her$ docker container remove container1
container1
```

, remove the container, and start a new container using
the same volume.
Verify whether the file still exists

```
mojorojo@mojorojo-vmware:~/dock-her$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
mojorojo@mojorojo-vmware:~/dock-her$ docker run -it --name container2 -v data_vol:/data ubuntu /bin/bash
root@98e99daae4ec:/# ls
bin  boot  data  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@98e99daae4ec:/# cd d
bash: cd: d: No such file or directory
root@98e99daae4ec:/# cd /data/hello.txt
bash: cd: /data/hello.txt: Not a directory
root@98e99daae4ec:/# cd /data/
root@98e99daae4ec:/data# ls
hello.txt
root@98e99daae4ec:/data# cat hello.txt
This is from Container 1
root@98e99daae4ec:/data# cd
root@98e99daae4ec:~# docker images
bash: docker: command not found
root@98e99daae4ec:~# exit
exit
```

# Q2. Dockerfile – Python Application

Create a simple Python program that prints a message.

Write a Dockerfile to build an image and run the Python script inside a container

```
mojorojo@mojorojo-vmware:~/dock-her$ cat python.py
print("Naman Bhai Jinda Hai")
mojorojo@mojorojo-vmware:~/dock-her$ cat Dockerfile
FROM python
WORKDIR /app
COPY python.py /app
CMD ["python","python.py"]
```

```
mojorojo@mojorojo-vmware:~/dock-her$ docker build -t  py .
[+] Building 31.3s (9/9) FINISHED                                                                                docker:default
 => [internal] load build definition from Dockerfile                                                                        0.0s
 => => transferring dockerfile: 110B                                                                                        0.0s
 => [internal] load metadata for docker.io/library/python:latest                                                            2.6s
 => [auth] library/python:pull token for registry-1.docker.io                                                               0.0s
 => [internal] load .dockerignore                                                                                           0.0s
 => => transferring context: 2B                                                                                             0.0s
 => [1/3] FROM docker.io/library/python:latest@sha256:63fdc2bca05b142ef64d05e8f9dbc4e8bf9f51294824db5c3d8b10f419fe1bdd  27.3s
 => => resolve docker.io/library/python:latest@sha256:63fdc2bca05b142ef64d05e8f9dbc4e8bf9f51294824db5c3d8b10f419fe1bdd   0.0s
 => => sha256:7732cb271831ee96d4d8089aaf42646f154246b86792007dfff35d29884dad99 250B / 250B                                 0.6s
 => => sha256:10c7fd96f0a80d9b8b22f277fda2fdacecacd0af2d56091fa428c18d885f021a 29.50MB / 29.50MB                           1.5s
 => => sha256:3f4c7bb8aa06776aadcd259e9406ea05dedcd9072136d9f1106f7b15e05b70d8 6.10MB / 6.10MB                             1.7s
 => => sha256:1ecbddc58afba44ec9b55fb51ec8bfdee5112cd31b39b564f5abb4567d094ffc 236.08MB / 236.08MB                         9.8s
 => => sha256:5397da1d1537c4d725f3090c5688a582e14eeaf7743d75d9b38bad1649554987 67.78MB / 67.78MB                           2.6s
 => => sha256:f33970743aee750df25f6c661132b7401c8fefe930e5f4803f4c8b6f567a6b55 25.62MB / 25.62MB                           2.7s
 => => sha256:a7730063fcfe708b222d34c4f07d633dfe087a28c05c4daaab2fa9943854c155 49.30MB / 49.30MB                           3.7s
 => => extracting sha256:a7730063fcfe708b222d34c4f07d633dfe087a28c05c4daaab2fa9943854c155                                  2.2s
 => => extracting sha256:f33970743aee750df25f6c661132b7401c8fefe930e5f4803f4c8b6f567a6b55                                  1.2s
 => => extracting sha256:5397da1d1537c4d725f3090c5688a582e14eeaf7743d75d9b38bad1649554987                                  3.2s
 => => extracting sha256:1ecbddc58afba44ec9b55fb51ec8bfdee5112cd31b39b564f5abb4567d094ffc                                 13.2s
 => => extracting sha256:3f4c7bb8aa06776aadcd259e9406ea05dedcd9072136d9f1106f7b15e05b70d8                                  0.8s
 => => extracting sha256:10c7fd96f0a80d9b8b22f277fda2fdacecacd0af2d56091fa428c18d885f021a                                  1.2s
 => => extracting sha256:7732cb271831ee96d4d8089aaf42646f154246b86792007dfff35d29884dad99                                  0.0s
 => [internal] load build context                                                                                           0.0s
 => => transferring context: 66B                                                                                            0.0s
 => [2/3] WORKDIR /app                                                                                                      1.0s
 => [3/3] COPY python.py /app                                                                                               0.0s
 => exporting to image                                                                                                      0.2s
```

```
mojorojo@mojorojo-vmware:~/dock-her$ docker container run py
Naman Bhai Jinda Hai
```

# Q3. Dockerfile – Java Application

Write a simple Java program that prints a message.
Create a Dockerfile to compile and run the Java program inside a container

```
mojorojo@mojorojo-vmware:~/dock-her-ja$ ls
Dockerfile  newjava.java  newj.java
mojorojo@mojorojo-vmware:~/dock-her-ja$ vim newjava.java
mojorojo@mojorojo-vmware:~/dock-her-ja$ docker build -t new_java .
[+] Building 2.3s (9/9) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 155B                                                                          0.0s
 => [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine                             1.1s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/4] FROM docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:bcc7ec7e8fef937ba9f01ee5f810361d722c6b5dbe19ac188  0.0s
 => => resolve docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:bcc7ec7e8fef937ba9f01ee5f810361d722c6b5dbe19ac188  0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 149B                                                                             0.0s
 => CACHED [2/4] WORKDIR /APP                                                                                 0.0s
 => [3/4] COPY NEWJAVA.JAVA /app                                                                              0.0s
 => [4/4] RUN JAVAC newjava.java                                                                              0.8s
 => exporting to image                                                                                        0.2s
 => => exporting layers                                                                                       0.1s
 => => exporting manifest sha256:b0a6990d6cfdc2b9bbab0bdcdc9456fef26ef90f77a9e036307b4e7039cf49c1             0.0s
 => => exporting config sha256:30b110444f4fc44516236ca88e2d271048f51104ec4d809c0eb50e55e934f133               0.0s
 => => exporting attestation manifest sha256:7ee1c703d7978e1750fc8f02a6c5ca1fbf4c6d46e1b7e87da252248bdcca064f  0.0s
 => => exporting manifest list sha256:2a2962af55335156716298d792ef50a20b377a8817033051b5b074cd0f4df7f6        0.0s
 => => naming to docker.io/library/new_java:latest                                                            0.0s
 => => unpacking to docker.io/library/new_java:latest                                                         0.1s
mojorojo@mojorojo-vmware:~/dock-her-ja$ docker run new_java
Kya aapke pass 11k hai?
```

# Q4. Docker Volume with MySQL

Run a MySQL container using a Docker volume for data storage.

```
mojorojo@mojorojo-vmware:~$ docker run -d --name sql5 -e MYSQL_ROOT_PASSWORD=acts -v data:/var/lib/mysql -p 5000:3306 mysql:8
9f8a4e0b79657e9831db6a8452d9192f9543edc558418dffc9e5978e728adc19
mojorojo@mojorojo-vmware:~$ docker exec -it sql5 mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Create a database and table, insert some records, then restart the container.

```
mysql> CREATE DATABASE cdac;
Query OK, 1 row affected (0.00 sec)

mysql> USE cdac;
Database changed
mysql> CREATE TABLE IF NOT EXISTS student(id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100), score INT);
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO student(name, score) VALUES('abhay',32);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT *  FROM student;
+----+-------+-------+
| id | name  | score |
+----+-------+-------+
|  1 | abhay |    32 |
+----+-------+-------+
1 row in set (0.00 sec)

mysql> exit
Bye
mojorojo@mojorojo-vmware:~$ docker container restart sql5
sql5
```

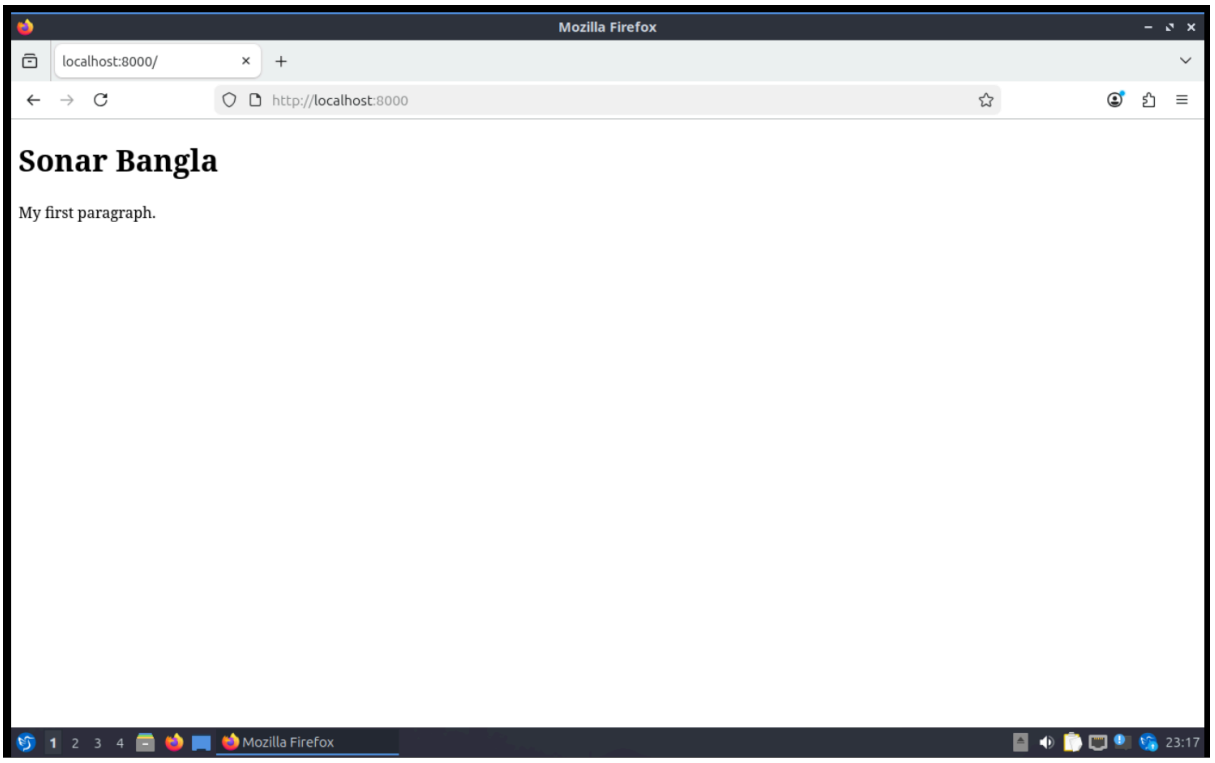
Verify whether the data persists after restart.

```
Bye
mojorojo@mojorojo-vmware:~$ docker container restart sql5
sql5
mojorojo@mojorojo-vmware:~$ docker exec -it sql5 mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE cdac
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> USE cdac;
Database changed
mysql> SELECT *  FROM student;
+----+-------+-------+
| id | name  | score |
+----+-------+-------+
|  1 | abhay |    32 |
+----+-------+-------+
```

## Q5. Dockerfile – Nginx Web Server

Create a custom HTML page.
Write a Dockerfile to serve this page using an Nginx container.
Run the container and verify the output in a browser

```
mojorojo@mojorojo-vmware:~$ mkdir html_web
mojorojo@mojorojo-vmware:~$ cd html_web/
mojorojo@mojorojo-vmware:~/html_web$ vim Dockerfile
mojorojo@mojorojo-vmware:~/html_web$ vim index.html
mojorojo@mojorojo-vmware:~/html_web$ docker build -t my-bangla .
[+] Building 0.6s (7/7) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 110B                                          0.0s
 => WARN: ConsistentInstructionCasing: Command 'From' should match the case   0.0s
 => [internal] load metadata for docker.io/library/nginx:latest               0.0s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [internal] load build context                                             0.0s
 => => transferring context: 132B                                             0.0s
 => [1/2] FROM docker.io/library/nginx:latest@sha256:7f0adca1fc6c29c8dc49a2e  0.1s
 => => resolve docker.io/library/nginx:latest@sha256:7f0adca1fc6c29c8dc49a2e  0.0s
 => [2/2] COPY INDEX.HTML /usr/share/nginx/html/index.html                    0.1s
 => exporting to image                                                        0.2s
 => => exporting layers                                                       0.1s
 => => exporting manifest sha256:71a4142c0735fdca2bb379d1df25f982e3a759a67a1  0.0s
 => => exporting config sha256:477ce9abce851999f1df0ef5f65167b9e4921305a7743  0.0s
 => => exporting attestation manifest sha256:670470558a525e81faadf8c9316eb17  0.0s
 => => exporting manifest list sha256:82f5457f489aa3aedd5028fc40c64ea4155150  0.0s
 => => naming to docker.io/library/my-bangla:latest                           0.0s
 => => unpacking to docker.io/library/my-bangla:latest                        0.0s

 1 warning found (use docker --debug to expand):
 - ConsistentInstructionCasing: Command 'From' should match the case of the command majority (uppercase) (line 1)
mojorojo@mojorojo-vmware:~/html_web$ docker run -d -p 8000:80 --name nginx-c my-bangla:latest
1b0e169dc8dfcf90fc07e0b7b7d3463de7b10a5ee5dd2488a487aa6ae10a307b
```




# Q6. Bind Mount vs Volume

Create a directory on the host system and store a file in it.

```
mojorojo@mojorojo-vmware:~$ mkdir Bindmt
mojorojo@mojorojo-vmware:~$ cd Bindmt/
mojorojo@mojorojo-vmware:~/Bindmt$ touch file
mojorojo@mojorojo-vmware:~/Bindmt$ ls /home/mojorojo/Bindmout
ls: cannot access '/home/mojorojo/Bindmout': No such file or directory
mojorojo@mojorojo-vmware:~/Bindmt$ ls /home/mojorojo/Bindmt/
file
```

Run a container using a bind mount to access this directory.

```
mojorojo@mojorojo-vmware:~/Bindmt$ docker run -itd --name ub1 -v /home/mojorojo/Bindmt:/home ubuntu
54e0e215fc90e91d33c88f33cc1acd70df39390ae57144922625e68b08597bd7
```

```
mojorojo@mojorojo-vmware:~/Bindmt$ docker exec -it ub1 /bin/bash
root@54e0e215fc90:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@54e0e215fc90:/# cd home/
root@54e0e215fc90:/home# ls
file
root@54e0e215fc90:/home#
```

Compare this with using a Docker volume and explain the difference.

**Bind Mounts** link a specific folder on your computer (like `~/Bindmt`) directly into the container. It's great for development because if you edit a file on your desktop, the container sees the change instantly.

**Volumes** are folders managed entirely by Docker in a hidden area. They are safer and faster for production because Docker handles the permissions and storage logic for you.

**The difference is that** Bind Mounts rely on your host's file structure; Volumes are independent and stay safe even if you move your project to another machine.

## Q7. Dockerfile – Shell Script Execution

Write a shell script that prints a message and system date.

```
mojorojo@mojorojo-vmware:~/bashdocker$ cat date.sh
#!/bin/sh

echo "This message displays the Date below ↓"
echo "--------------------------------------"
date -I
echo "--------------------------------------"
mojorojo@mojorojo-vmware:~/bashdocker$
```

Create a Dockerfile to execute this script inside a container.

```
mojorojo@mojorojo-vmware:~/bashdocker$ docker build -t data .
[+] Building 2.0s (8/8) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 108B                                                                          0.0s
 => [internal] load metadata for docker.io/library/alpine:3.22.4                                              1.8s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/3] FROM docker.io/library/alpine:3.22.4@sha256:310c62b5e7ca5b08167e4384c68db0fd2905dd9c7493756d356e893909057601   0.0s
 => => resolve docker.io/library/alpine:3.22.4@sha256:310c62b5e7ca5b08167e4384c68db0fd2905dd9c7493756d356e893909057601   0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 195B                                                                             0.0s
 => CACHED [2/3] WORKDIR /APP                                                                                 0.0s
 => [3/3] COPY DATE.SH /APP                                                                                   0.0s
 => exporting to image                                                                                        0.1s
 => => exporting layers                                                                                       0.0s
 => => exporting manifest sha256:a769f1de576973960c2c186ecffffdc55dbe9a9205b39c0a3a7b9338dfc9b6d9             0.0s
 => => exporting config sha256:944b6cf1831a46de9d0f76e2141fe570156cee096afcadf3f2e4786582a313de               0.0s
 => => exporting attestation manifest sha256:113d1a3b192d2be894036656bd0177638e0d256685e1adfdc19d1bfdc592ad82  0.0s
 => => exporting manifest list sha256:e59eaafe084947b2873bdbe1a772d6df62e3e16f1762e492a43f6039210bad0e         0.0s
 => => naming to docker.io/library/data:latest                                                                0.0s
 => => unpacking to docker.io/library/data:latest                                                             0.0s
mojorojo@mojorojo-vmware:~/bashdocker$ docker run -it --name date2 data:latest
This message displays the Date below ↓
--------------------------------------
2026-04-18
--------------------------------------
mojorojo@mojorojo-vmware:~/bashdocker$ docker run  data
This message displays the Date below ↓
--------------------------------------
2026-04-18
```

## Q8. Shared Volume Between Containers

Create a Docker volume and use it with two different containers.
Write data from one container and access the same data from another container.
Verify that both containers can share the same data

```
mojorojo@mojorojo-vmware:~/bashdocker$ docker volume create v1
v1
mojorojo@mojorojo-vmware:~/bashdocker$ cd ..
mojorojo@mojorojo-vmware:~$ mkdir for2containers
mojorojo@mojorojo-vmware:~$ cd for2containers/
mojorojo@mojorojo-vmware:~/for2containers$ cat > file1

from container 1
^C
mojorojo@mojorojo-vmware:~/for2containers$ docker run -it --name container1 -v v1:/home/mojorojo/for2containers ubuntu /bin/ba
sh
root@0cfdb6f63365:/# cd home/
root@0cfdb6f63365:/home# ls
mojorojo  ubuntu
root@0cfdb6f63365:/home# cd mojorojo/'
> ^C
root@0cfdb6f63365:/home# cd mojorojo/
root@0cfdb6f63365:/home/mojorojo# ls
for2containers
root@0cfdb6f63365:/home/mojorojo# cd for2containers/
root@0cfdb6f63365:/home/mojorojo/for2containers# ls
root@0cfdb6f63365:/home/mojorojo/for2containers# echo "This file is in Container 1" > con
root@0cfdb6f63365:/home/mojorojo/for2containers# exit
exit
```

```
mojorojo@mojorojo-vmware:~/for2containers$ docker run -it --name container2 -v v1:/home/mojorojo/for2containers ubuntu /bin/ba
sh
root@a72de97e823d:/# cd m
media/ mnt/
root@a72de97e823d:/# cd home/
root@a72de97e823d:/home# ls
mojorojo  ubuntu
root@a72de97e823d:/home# cd mojorojo/
root@a72de97e823d:/home/mojorojo# ls
for2containers
root@a72de97e823d:/home/mojorojo# cd for2containers/
root@a72de97e823d:/home/mojorojo/for2containers# ls
con
root@a72de97e823d:/home/mojorojo/for2containers# cat con
This file is in Container 1
root@a72de97e823d:/home/mojorojo/for2containers#
```